

Week 2: Product and Category Management System

Scenario:

The e-commerce company wants to maintain information about products and their categories. Each product belongs to a specific category, and the system should manage product details such as name, price, and stock availability.

1. Design Product and Category Tables

Category Table:

```
SQL> CREATE TABLE Category(Category_ID VARCHAR2(7),
 2 Category_Name VARCHAR2(20),
 3 Description VARCHAR2(50));

Table created.
```

Product Table:

```
SQL> CREATE TABLE Product(Product_ID VARCHAR2(7),
 2 Product_Name VARCHAR2(20),
 3 Category_ID VARCHAR2(7),
 4 Price NUMBER(9,2),
 5 Stock_Quantity Number);

Table created.
```

The two tables for Category and Product have been created.

Now, we have to find the relationship between the tables and apply constraints to the tables.

2. Define Relationships and Constraints

Relationships:

One Category Can have many Products.

The relationship between Category and Product is One to many (1-> *)

We found that, what is their relationship. Now we have to discuss that which is the key that is going to make this relationship.

Before that, we need to understand about KEY Constraints.

PRIMARY KEY: The Column(s) or Field(s) that uniquely identifies a row/ tuple

In these two tables, which field can be a Primary Key?

In Category Table, Category_ID is a unique Value that helps to identify each category. Likewise, in Product Table Product_ID is a unique Column.

FOREIGN KEY: The Key which links two tables.

Therefore, Category_ID acts as Foreign Key in the Product Table.

We can explore some other constraints like NOT NULL, CHECK and UNIQUE

NOT NULL: Ensures that the Column cannot have a empty (null) value.

For Example, a Product name or Category name cannot be empty. All category or product must have a name.

CHECK: Validates the values in a column satisfies the condition.

For Example, To check the Price of a Product must always be a positive number.

UNIQUE: Ensures that there is no repeated value for a column.

For Example, If we mention UNIQUE Constraint to Category Name, no two category can have same name.

Defining Relationship and Creating Constraints for Product and Category:

Primary Key:

```
SQL> ALTER TABLE Category
2 ADD CONSTRAINT PK_Category PRIMARY KEY (Category_ID);

Table altered.

SQL> ALTER TABLE Product
2 ADD CONSTRAINT PK_Product PRIMARY KEY (Product_ID);

Table altered.
```

Creating Relationship using FOREIGN KEY:

```
SQL> ALTER TABLE Product
2 ADD CONSTRAINT FK_Product_Category
3 FOREIGN KEY (Category_ID) REFERENCES Category(Category_ID);

Table altered.
```

Adding Constraints:

NOT NULL:

```
SQL> ALTER TABLE Category
2 MODIFY Category_Name VARCHAR(100) NOT NULL;

Table altered.
```

```
SQL> ALTER TABLE Product
2 MODIFY Product_Name VARCHAR(150) NOT NULL;

Table altered.
```

CHECK:

```
SQL> ALTER TABLE Product
  2  ADD CONSTRAINT CHK_Price CHECK (Price > 0);

Table altered.

SQL> ALTER TABLE Product
  2  ADD CONSTRAINT CHK_Stock CHECK (Stock_Quantity >= 0);

Table altered.
```

UNIQUE:

```
SQL> ALTER TABLE Category
  2  ADD CONSTRAINT UQ_Category_Name UNIQUE (Category_Name);

Table altered.
```

3. Insert Values:

```
SQL> INSERT INTO Category
  2  VALUES(
  3  '&Category_ID',
  4  '&Category_Name',
  5  '&Description');
Enter value for category_id: Cat0001
old 3: '&Category_ID',
new 3: 'Cat0001',
Enter value for category_name: Electronics
old 4: '&Category_Name',
new 4: 'Electronics',
Enter value for description: Electronic gadgets and devices
old 5: '&Description')
new 5: 'Electronic gadgets and devices')

1 row created.

SQL> /
Enter value for category_id: Cat0002
old 3: '&Category_ID',
new 3: 'Cat0002',
Enter value for category_name: Clothing
old 4: '&Category_Name',
new 4: 'Clothing',
Enter value for description: Men,Women and Kids Apparels
old 5: '&Description')
new 5: 'Men,Women and Kids Apparels')

1 row created.
```

Product Table:

```
SQL> INSERT INTO Product
  2 VALUES(
  3 '&ProductID',
  4 '&Product_Name',
  5 '&CategoryID',
  6 &Price,
  7 &Stock_Quantity);
Enter value for productid: P1001
old 3: '&ProductID',
new 3: 'P1001',
Enter value for product_name: Laptops
old 4: '&Product_Name',
new 4: 'Laptops',
Enter value for categoryid: Cat0001
old 5: '&CategoryID',
new 5: 'Cat0001',
Enter value for price: 67500
old 6: &Price,
new 6: 67500,
Enter value for stock_quantity: 2
old 7: &Stock_Quantity)
new 7: 2)

1 row created.
```

Like this insert few more rows.

After Insertion Category Table:

```
SQL> SELECT * FROM Category;

CATEGOR CATEGORY_NAME
-----
DESCRIPTION
-----
Cat0001 Electronics
Electronic gadgets and devices

Cat0002 Clothing
Men,Women and Kids Apparels

Cat0003 Home
Household Appliances

CATEGOR CATEGORY_NAME
-----
DESCRIPTION
-----
Cat0004 Books
All Genre of Books
```

Product Table

```
SQL> SELECT * FROM Product;
```

PRODUCT

PRODUCT_NAME

CATEGOR PRICE STOCK_QUANTITY

P1001

Laptops

Cat0001 67500 2

P1002

Smart Phones

Cat0001 25000 5

PRODUCT

PRODUCT_NAME

CATEGOR PRICE STOCK_QUANTITY

P2001

Shoes

Cat0002 1400 5

P2002

Saree

PRODUCT

PRODUCT_NAME

CATEGOR PRICE STOCK_QUANTITY

Cat0002 499 10

P3001

Television

Cat0003 42000 2

P4001

PRODUCT

PRODUCT_NAME

CATEGOR PRICE STOCK_QUANTITY

Dune

Cat0004 599 4

P4002

Thulasimadam

Cat0004 250 2

7 rows selected.

CRUD Operations:

Insert Record:

```
SQL> INSERT INTO Product (Product_ID,Product_Name, Category_ID, Price, Stock_Quantity)
  2  VALUES ('P3002','Washing Machine', 'Cat0003', 18000, 12);

1 row created.
```

Update Price:

```
SQL> UPDATE Product SET Price = 60000 WHERE Product_Name = 'Laptops';

1 row updated.
```

Delete a Product:

```
SQL> DELETE FROM Product WHERE Product_Name = 'Thulasimadam';

1 row deleted.
```

5. Generate Category-wise Product Reports

Write SQL queries to:

- Display all products under each category.

```
SQL> SELECT c.Category_Name, p.Product_Name, p.Price, p.Stock_Quantity
2 FROM Product p JOIN Category c ON p.Category_ID = c.Category_ID;
```

```
CATEGORY_NAME
```

```
PRODUCT_NAME
```

```
PRICE STOCK_QUANTITY
```

```
Electronics
```

```
Laptops
```

```
60000
```

```
2
```

```
Electronics
```

```
Smart Phones
```

```
25000
```

```
5
```

```
CATEGORY_NAME
```

```
PRODUCT_NAME
```

```
PRICE STOCK_QUANTITY
```

```
Clothing
```

```
Shoes
```

```
1400
```

```
5
```

```
Clothing
```

```
Saree
```

```
CATEGORY_NAME
```

```
PRODUCT_NAME
```

```
PRICE STOCK_QUANTITY
```

```
499
```

```
10
```

```
Home
```

```
Television
```

```
42000
```

```
2
```

```
Books
```

```
CATEGORY_NAME
```

```
PRODUCT_NAME
```

```
PRICE STOCK_QUANTITY
```

```
Dune
```

```
599
```

```
4
```

```
Home
```

```
Washing Machine
```

```
18000
```

```
12
```

```
7 rows selected.
```

- Count number of products in each category.

```
SQL> SELECT c.Category_Name, COUNT(p.Product_ID) AS Product_Count
2 FROM Product p JOIN Category c ON p.Category_ID = c.Category_ID
3 GROUP BY c.Category_Name;
```

```
CATEGORY_NAME
```

```
PRODUCT_COUNT
```

```
Electronics
```

```
2
```

```
Clothing
```

```
2
```

```
Home
```

```
2
```

```
Books
```

```
1
```

- Find highest-priced product in each category.

```
SQL> SELECT c.Category_Name, p.Product_Name, p.Price AS Highest_Price
2 FROM Product p
3 JOIN Category c ON p.Category_ID = c.Category_ID
4 WHERE p.Price = (
5     SELECT MAX(p2.Price)
6     FROM Product p2
7     WHERE p2.Category_ID = p.Category_ID
8 );
```

```
CATEGORY_NAME
```

```
-----
```

```
PRODUCT_NAME
```

```
-----
```

```
HIGHEST_PRICE
```

```
-----
```

```
Electronics
```

```
Laptops
```

```
60000
```

```
Clothing
```

```
Shoes
```

```
1400
```

```
CATEGORY_NAME
```

```
-----
```

```
PRODUCT_NAME
```

```
-----
```

```
HIGHEST_PRICE
```

```
-----
```

```
Home
```

```
Television
```

```
42000
```

```
Books
```

```
Dune
```

```
CATEGORY_NAME
```

```
-----
```

```
PRODUCT_NAME
```

```
-----
```

```
HIGHEST_PRICE
```

```
-----
```

```
599
```

- Display categories having more than 5 products.

```
SQL> SELECT c.Category_Name
2 FROM Product p JOIN Category c ON p.Category_ID = c.Category_ID
3 GROUP BY c.Category_Name HAVING COUNT(p.Product_ID) > 5;
```

```
CATEGORY_NAME
```

```
-----
```

```
Electronics
```

- Find average product price category-wise.

```
SQL> SELECT c.Category_Name, AVG(p.Price) AS Avg_Price
2 FROM Product p JOIN Category c ON p.Category_ID = c.Category_ID
3 GROUP BY c.Category_Name;
```

```
CATEGORY_NAME
```

```
AVG_PRICE
```

```
-----
```

```
Electronics
```

```
32958.1667
```

```
Clothing
```

```
949.5
```

```
Home
```

```
30000
```

```
Books
```

```
599
```